

O b s e r v e r

Observer Pattern

Notify dependents on change

DEFINITION

Define a one-to-many dependency so that when one object changes state, all its dependents are notified.

Observer wires a one-to-many dependency: a subject keeps a list of observers and, when its state changes, notifies them all. Subject and observers are coupled only through a thin `update()` interface, so the subject broadcasts without knowing who is listening, and listeners come and go at runtime.

WHY IT MATTERS

When a change in one object must update an unknown number of others, hard-wiring the calls couples the subject to every dependent and breaks each time the set changes. Observer inverts that: the subject talks to an interface and fans out one `notify()` to all registered observers, so you add listeners without touching it (OCP). The classic cost is the lapsed-listener leak — observers that forget to unsubscribe — plus hard-to-trace update cascades.

— How it works

Subject	Keeps the observer list — attach / detach / notify.
Observer	The update() interface every dependent implements.
ConcreteObserver	Holds a reference, reacts, and syncs to the subject's state.

HOW TO APPLY

1. Give the subject attach / detach and a notify() that loops its observers.
2. Have observers implement one update() (push the data, or let them pull it).
3. Subscribe at setup and, crucially, detach on teardown to avoid leaks.
4. Guard each callback so one throwing observer can't starve the rest.

LITMUS TEST

Does a change in one object need to update an unknown, changing set of others? If you don't know how many dependents up front, broadcast through an observer list.

— Smell → fix

The subject calls each dependent by name — welded to all of them:

```
class Cart {  
  addItem(x) {  
    badge.refresh() // hard-wired dependents  
    analytics.track(x) // add one → edit Cart  
  }  
}
```

↓ BROADCAST TO A LIST OF OBSERVERS

```
type Observer = (c: Cart) => void  
class Cart {  
  #obs = new Set<Observer>()  
  subscribe(o: Observer) { this.#obs.add(o) }  
  addItem(x) { this.total += x.price; this.#notify() }  
  #notify() { this.#obs.forEach(o => o(this)) }  
}
```

Cart now notifies a list it never had to name; badge and analytics subscribe and can be added or removed without editing Cart. Pair every subscribe with an unsubscribe on teardown, or you leak the observers.

USE WHEN

- ✓ Event systems
- ✓ Reactive UI / state; model-view sync

AVOID WHEN

- ▲ Complex update cascades — prefer an event bus / reactive streams

— Trade-offs & pitfalls

PROS	CONS
+ Loose coupling + Dynamic subscribers	– Update storms – Memory leaks if not detached – Order undefined

COMMON PITFALLS

- ▲ Lapsed-listener leak: observers that never unsubscribe keep the subject (and themselves) alive — always pair subscribe with teardown.
- ▲ Don't rely on notification order; assuming one makes behavior fragile and registration-dependent.
- ▲ A blind notify can trigger wide cascades or re-entrancy; an unguarded loop lets one throwing observer skip the rest — isolate each callback.

WHERE YOU SEE IT

- Node EventEmitter (.on / .emit) and DOM addEventListener; RxJS Observables and Subjects.
- Fine-grained reactive signals (SolidJS, Vue, Angular) auto-tracking deps and batching updates.
- Model-view sync in MVVM and reactive UI.

SEE ALSO

Mediator	centralizes routing through a hub; Observer is decentralized one-to-many broadcast
Command	an observer's reaction is often a Command the subject fires
MVVM	the View observes the ViewModel — Observer is the binding mechanism
Low Coupling	Observer is a textbook low-coupling technique — talk through an interface

— Interview & sources

Observer vs Pub/Sub?

Observer's subject holds direct references to its observers; Pub/Sub routes through a broker / channel so publisher and subscriber never know each other.

Push or pull?

Push sends the changed data in the notification; pull lets observers query what they need. Push is simpler, pull is more flexible.

Most common bug?

The lapsed-listener leak — observers that never unsubscribe keep the subject (and themselves) alive. Always pair subscribe with teardown.

What if an observer throws?

In a naive loop it aborts the rest. Wrap each callback, or deliver via a queue, so one failure can't starve other observers.

REMEMBER

One subject's change is broadcast to many observers automatically — coupled only through a thin interface.

SOURCES

GoF — "Design Patterns" (1994): Observer (behavioral)

Freeman & Robson — "Head First Design Patterns" (2nd ed., 2020): Observer